

# Mini EHR Portal

## Advanced Healthcare Portal & AI Diagnostics

---

Technical System Design, Database Schema & Presentation Guide  
Designed for Patients & Healthcare Providers

*Created for Professional Slides & Project Showcase*  
*Generates formatted PDF report containing all system components*

## 1. Executive Summary

---

Mini EHR is an Electronic Health Record web application designed for patients and medical practitioners. The goal of the project is to provide a single, secure environment where patients track their chronic conditions, upload clinical documents, and get real-time medical updates, while doctors track assigned patient records and respond to urgent clinical events.

The app incorporates AI capabilities to decode and understand complex, physical medical charts and prescriptions via visual LLM analysis, converting unstructured images into clear instructions and tracking lists.

## 2. High-Level Product Features

---

### Patient Portal Features:

- o **Dashboard Snapshots:** Review and graph historical blood sugar fluctuations over time.
- o **Symptom Check-ins:** Log symptom descriptions, durations, severity values, location, food logs, and attach images.
- o **AI Prescription Scanner:** Upload handwriting or print prescriptions. Returns medicine lists, instructions, and side effects.
- o **Clinical Reports Vault:** Secure space to save lab PDFs and files with browser-integrated file view previews.
- o **Medication Reminders:** Set schedules, check-in appointments, and organize tasks.

### Doctor Portal Features:

- o **My Connected Patients:** View and search patient files linked to the doctor's registration identifier code.
- o **Patient Detail Timeline:** Deep-dive view showing chronological clinical check-ins, medication logs, and trends.
- o **Prescription Writer:** Draft, customize, and issue new digital prescriptions logged to the DB.
- o **Urgent Patient Incident Alerts:** Monitors real-time alerts flagged by patient high-severity entries. Direct action panel to input clinical advice and resolve alerts.

### 3. System Architecture & Component Mapping

---

Mini EHR is built using a decoupled multi-tier web application architecture. The components communicate asynchronously using RESTful HTTP requests.

- o **Client Tier (Frontend):** Built with React 18 and Vite. Styled with a premium dark-themed glassmorphism system using custom CSS variable tokens. Charts are rendered using SVGs via Recharts, and icons are mapped using Lucide React.
- o **Gateway (Proxy):** Nginx web server containers reverse-proxy API requests from the browser directly to the FastAPI server, serving static React bundles efficiently in production.
- o **Application Tier (Backend):** FastAPI web services utilizing Pydantic for validation schemas. Includes automated password hashing (bcrypt), Gmail SMTP interfaces for password recovery, and multi-thread SQL operations.
- o **Data Tier (Database):** PostgreSQL database implementing custom connection pools (psycopg2-binary). Fast retrieval speeds are guaranteed via database indexes created on primary query keys (patient\_id, doctor\_id).
- o **AI Core Services:** Groq Cloud API integration routing request payloads to Llama 4 Scout. Images are processed as base64-encoded strings for instant entity-extraction.

### 4. Database Schema and Structure

---

The system operates on 7 relational tables with constraints and indexes to enforce data integrity:

- o **doctors:** Stores doctor profiles. Each doctor is issued a unique string 'doctor\_code' at signup (e.g. DOC-B91) which is used for binding patients.
- o **patients:** Stores patient details. Connects to doctor via foreign key 'doctor\_id' reference.
- o **symptoms:** Maintains patient logs. Stores description, severity (1-10 slider), occurrence date, blood sugar metrics, notes, meal information, and base64 preview files.
- o **reports:** Stores clinical reports (PDF and images) uploaded by patients or assigned doctors.
- o **prescriptions:** Stores digital medication instructions issued by doctors.
- o **prescription\_scans:** Stores AI scanner histories using JSONB columns to parse medicine arrays.
- o **user\_settings:** Maintains notification preferences (e.g., alerts\_enabled toggle).

## 5. 'Inch-by-Inch' Feature Workflows

---

### A. Secure Authentication & Registration

1. The user inputs their email. The system sends a POST request to '/auth/send-otp' which creates a 6-digit random verification code in the server memory and emails it using Python's smtplib via Gmail TLS SMTP (Port 587).
2. Once verified, the user signs up. Passwords are encrypted using bcrypt. Salted and hashed passwords are saved to PostgreSQL (never in plain text).
3. If registering as a doctor, the backend auto-generates a unique DOC-XXXX code, allowing patients to join their network.

### B. Intelligent AI Prescription Scanner

1. The patient drops a prescription image. The web app converts the image file locally to a base64 Data URI.
2. The base64 string is POSTed to '/ai/scan-prescription' along with the patient's ID.
3. The backend strips the header and passes the image to the Groq API (llama-4-scout-17b) with strict prompt instructions: ignore patient names/insurance details to protect privacy, extract medicines, descriptions, and side effects, and format as strict JSON.
4. The JSON is validated. If the base64 string is under 500KB, it is stored in 'prescription\_scans' database records, enabling patients to review their history later.

### C. Urgent Patient Alerts & Incident Response

1. The patient fills out a symptom card. If they choose to alert their doctor, they input an alert message, calling the POST '/alerts' API.
2. The incident log is generated. The doctor's dashboard queries '/alerts/doctor/{doctor\_id}' and renders a glowing warning block under the 'Urgent Incident Board'.
3. The doctor inputs clinical advice and submits. The request hits PUT '/alerts/{alert\_id}/comment'. The system updates the DB and flags the incident as resolved.
4. The patient dashboard pulls this data from the API and displays the doctor's comment in a glowing banner.

## 6. Presentation Slides Structure (Slide-by-Slide)

---

- o **Slide 1: Title Slide** Project name, subtitle, presenter credentials.
- o **Slide 2: Problems in Modern EHRs** Prescription errors, lack of Condition Timeline tools, delayed emergency alert loops.
- o **Slide 3: Project Solution** Glassmorphic dashboards, AI prescription interpreter, real-time alert feedbacks.
- o **Slide 4: Patient Portal** Symptom check-ins (severity, blood sugar, photos), analytics graphs, PDF reports.
- o **Slide 5: Doctor Portal** Connected patient list, medical timelines, direct prescription tool, alert actions.
- o **Slide 6: Architecture Overview** Decoupled React (Vite) client, Nginx gateway, FastAPI (Python), PostgreSQL DB.
- o **Slide 7: Database Relational Design** ER diagram, tables, indexes for quick search, JSONB columns for AI scan records.
- o **Slide 8: The Vision AI Scanner** How base64 images are processed, prompt engineering with Groq, JSON parsing, GDPR-compliant privacy filters.
- o **Slide 9: Security and Code Details** Passwords hashed via bcrypt, SSL pools, hidden keys in .env, Nginx proxy configs.
- o **Slide 10: Summary & Next Steps** A sleek, responsive EHR. Next steps: WebRTC consulting, automated push alerts.

*This PDF document covers all key features of Mini EHR. Use the markdown file in your workspace to generate presentation slides or view interactive graphs.*